

MetrCheck AI

End-to-End System Working, Architecture & Frontend-Backend Connectivity Guide
Smart India Hackathon • Problem Statement: SIH26034

1. Executive Summary & Quick Working Overview (In Short)

Core Concept: MetrCheck AI automatically audits packaged commodity labels against Indian **Legal Metrology (2011)** and **FSSAI** labelling regulations using Deep Learning Optical Character Recognition (OCR), Regex Heuristic Extraction, and a 13-rule Codified Statutory Matrix.

Short 3-Point Connectivity:

- **Frontend (React 19 on Port 5173):** Captures multi-panel label images, issues asynchronous REST API requests via `fetch()`, and renders interactive bounding box overlays.
- **Backend (FastAPI on Port 8000):** Ingests multipart files, assesses image quality with OpenCV, detects text coordinates with **PaddleOCR (PP-OCRv4)**, executes 13 legal rules, computes compliance scores (0-100), and returns structured JSON.
- **Database (SQLite via aiosqlite):** Persistently stores full inspection records, bounding box coordinates, and audit history.

2. Complete Technology Stack Matrix

Layer	Technology Used	Functional Role & Purpose
Frontend UI	React 19, TypeScript, Vite, Tailwind CSS v4	Responsive Single-Page Web App with multi-slot image upload and Light/Dark mode.
Backend API	FastAPI (Python 3.11), Uvicorn, Pydantic v2	High-speed asynchronous REST API with strict request/response data schemas.
Primary OCR	PaddleOCR (PP-OCRv4: DBNet + SVTR)	Deep learning detection, angle classification, text recognition & 2D bounding boxes.
Image Quality	OpenCV (cv2) & Pillow (PIL)	Laplacian blur variance score, brightness/glare, and resolution quality screening.
Extraction	Compiled Regex + Heuristics + Typo Repair	Isolates statutory entities (MRP, Net Qty, Dates, FSSAI Lic No, Address, Phone/Email).
Rules Engine	Declarative Python Rule Matrix (13 Rules)	Codifies 8 Legal Metrology + 5 FSSAI rules into deterministic compliance verdicts.
Database	SQLite (aiosqlite)	Non-blocking local relational storage for inspection history and audit tracking.
Data Viz	Recharts & Lucide React	Score gauge circles, history trend distribution, and clear iconography.

3. How Backend & Frontend Are Connected (Detailed Architecture)

Communication between the React frontend (running on port 5173) and FastAPI backend (running on port 8000) operates strictly over **HTTP/REST**. The key connection components include:

- **A. Central Fetch Client (frontend/src/services/api.ts):** All UI interactions trigger typed functions in `api.ts`. The base URL points to `http://localhost:8000/api`. Requests are sent as either JSON payloads or `multipart/form-data`.
- **B. Cross-Origin Resource Sharing (CORS Middleware):** In `backend/main.py`, `CORSMiddleware` explicitly whitelists origins (`http://localhost:5173`), enabling browser fetch calls without cross-origin blocking.
- **C. Strict Data Schema Mirroring (Type Safety):** Frontend TypeScript interfaces in `frontend/src/types/index.ts` mirror Backend Pydantic models in `backend/models/schemas.py`, ensuring complete consistency across all fields (`score`, `status`, `checks`, `bbox`, `detected_value`).

- **D. Static Asset Serving:** Uploaded label images and generated thumbnails are served directly via FastAPI static mount: `app.mount('/uploads', StaticFiles(...))` so the frontend Evidence Viewer can stream high-res images.

Frontend Page & Backend Endpoint Mapping Matrix

Frontend Page	UI Route	Backend API Endpoint	Data Exchange & Purpose
Dashboard	/	GET /api/history/stats GET /api/history	Loads scan metrics, status distributions, and recent inspections.
Analyze	/analyze	POST /api/analyze	Transfers multipart images & perspective labels; receives full audit JSON.
Results	/results/:id	GET /api/history/:id GET /api/report/:id/download	Renders score gauge, 13-rule checks, bounding box highlights, and PDF export.
Demo Cases	/demo	GET /api/demo/:id	Fetches pre-calculated benchmark datasets (Compliant, Warning, Violation).
History	/history	GET /api/history DELETE /api/history/:id	Displays historical scan audit table and allows record management.
Rules Explorer	/rules	GET /api/rules	Retrieves active Legal Metrology and FSSAI rules registry definitions.

4. End-to-End Processing Lifecycle (From Image Upload to Audit Dossier)

When an inspector uploads packaging label artwork, the system executes the following 7 deterministic steps:

- **Step 1: Multi-Slot Ingestion (Frontend):** The user selects/drops label images on Analyze.tsx across 4 dedicated slots: *Front, Back, Side 1, Side 2*. The frontend checks image types (JPG, PNG, WebP) and size limits ($\leq 10\text{MB}$), packages them into a FormData object, and issues a POST /api/analyze request.
- **Step 2: File Storage & Image Quality Screening (Backend):** image_service.py stores images in /uploads with UUID filenames. OpenCV (quality.py) calculates Laplacian variance to detect camera blur (< 70 threshold), checks brightness (< 45 underexposed, > 230 glare), and verifies resolution ($> 400\text{px}$). Quality warnings are attached to the audit record.
- **Step 3: Deep Learning OCR & Coordinate Tracking: PaddleOCR (PP-OCRv4)** runs DBNet text detection, angle classification (for rotated packaging text), and SVTR text recognition. It records exact 2D spatial bounding box coordinates [xmin, ymin, xmax, ymax] and recognition confidence scores for every text line.
- **Step 4: Typo Repair & Entity Extraction:** repair.py repairs packaging-specific optical misrecognitions (e.g. '400 9' $\rightarrow$ '400 g', 'FSSAI Lic O12...' $\rightarrow$ '10012...'). Compiled regex patterns in patterns.py parse mandatory fields: MRP, standard metric Net Quantity, Dates, FSSAI Number, Manufacturer & PIN Code, Country of Origin, and Customer Care.
- **Step 5: Codified Statutory Rule Verification:** The 13-rule compliance engine (compliance/engine.py) evaluates extracted data against 8 Legal Metrology and 5 FSSAI rules, assigning outcomes: **PASS, WARNING, NEEDS_REVIEW, FAIL**, or **NOT_APPLICABLE**.
- **Step 6: Scoring Algorithm & Prioritized Recommendations:** scorer.py computes a normalized 0–100 score: $(1.0*\text{PASS} + 0.85*\text{REVIEW} + 0.5*\text{WARN} + 0.0*\text{FAIL}) / \text{Applicable} * 100$. recommendations.py maps every non-compliant finding to its exact statutory clause and corrective action. The complete record is saved to SQLite via aiosqlite.
- **Step 7: Interactive Results & Visual Bounding Boxes (Frontend):** FastAPI returns the JSON response. React navigates to /results/:id. EvidenceViewer.tsx draws an interactive SVG overlay over the product image. When the user hovers over any finding, the exact text bounding box illuminates with color-coded highlighting.

5. Codified Statutory Rules Matrix (All 13 Rules)

Rule ID	Statute & Reference	Rule Title & Requirement Description	Severity
LM-001	Rule 6(1)(a)	Manufacturer / Packer / Importer: Name & physical address with PIN code.	HIGH
LM-002	Rule 6(1)(b)	Generic / Common Name: Commodity name on Principal Display Panel.	HIGH
LM-003	Rule 6(1)(c) & 12	Net Quantity & Units: Standard metric unit (g, kg, ml, L) & font height.	HIGH
LM-004	Rule 6(1)(e)	Maximum Retail Price (MRP): Formatted in Rs/₹ with 'incl. of all taxes'.	HIGH
LM-005	Rule 6(1)(f)	Consumer Care: Mandatory phone helpline number and/or email address.	HIGH
LM-006	Rule 6(10)	Country of Origin: Mandatory origin declaration for imported/domestic goods.	MEDIUM
LM-007	Rule 6(11)	Unit Sale Price (USP): Price per gram or milliliter for retail goods.	MEDIUM
LM-008	Rule 6(1)(d)	Mfg / Packing Date: Conspicuous month & year (MM/YYYY) marking.	HIGH
FS-001	Regulation 5(4)	FSSAI Logo & Licence: Valid 14-digit numeric FSSAI registration.	HIGH
FS-002	Regulation 5(1)	Name of Food: Standard food category & true nature identification.	HIGH
FS-003	Regulation 5(2)	Ingredients List: Descending order ingredients & allergen warnings.	MEDIUM
FS-004	Regulation 5(3)	Nutritional Information: Energy, Protein, Carb, Fat per 100g/100ml.	HIGH
FS-005	Regulation 5(7)	Date Marking: 'Best Before' or 'Expiry Date' with storage instructions.	HIGH

6. Quick Execution & Server Endpoints

- **Frontend Dashboard:** <http://localhost:5173> (React 19 + TypeScript + Vite)
- **Backend API Server:** <http://localhost:8000> (FastAPI + Uvicorn + PaddleOCR)
- **Interactive API Docs:** <http://localhost:8000/docs> (Swagger UI / OpenAPI 3.0)
- **Health Check Endpoint:** <http://localhost:8000/api/health> (Status: Healthy)
- **One-Click Startup:** Execute `run.bat` to launch both servers simultaneously.